Advanced Computer Architecture LAB
Autumn Semester 2025-26
Assignment 6

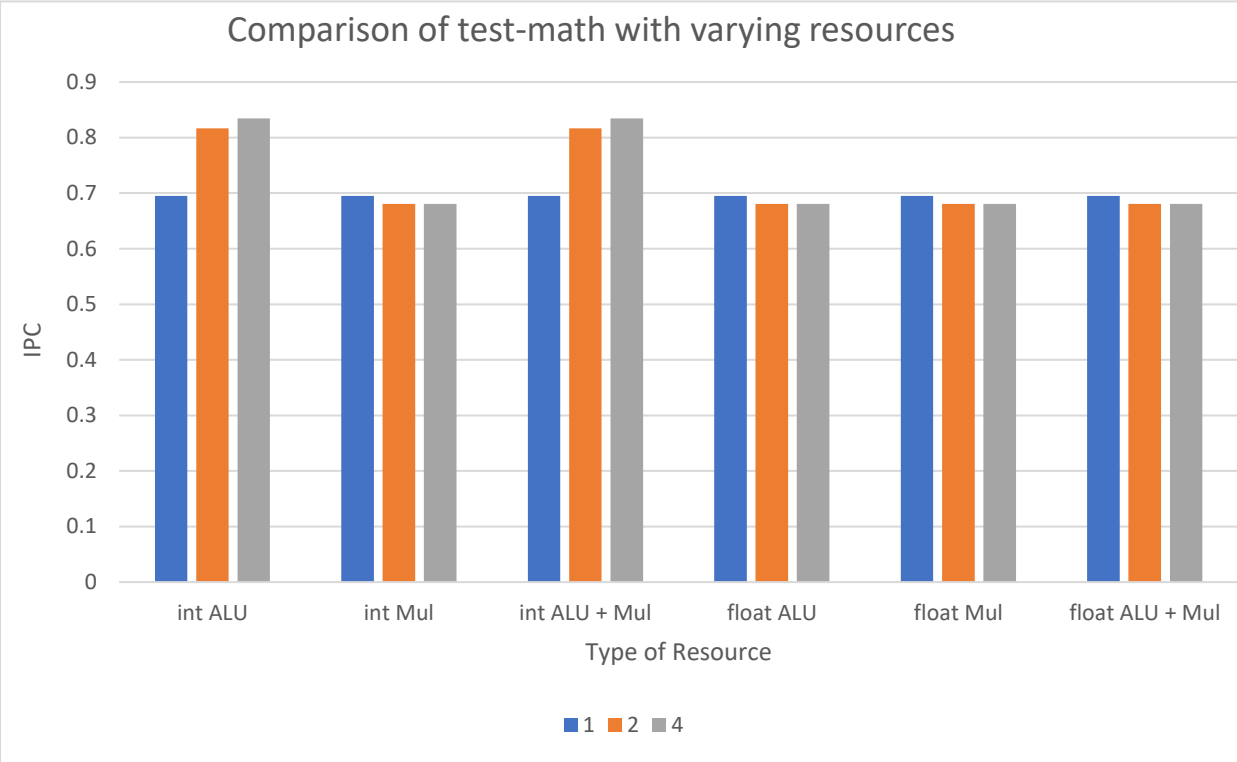
Name – Soham Chakraborty
Roll No – 22CS02002

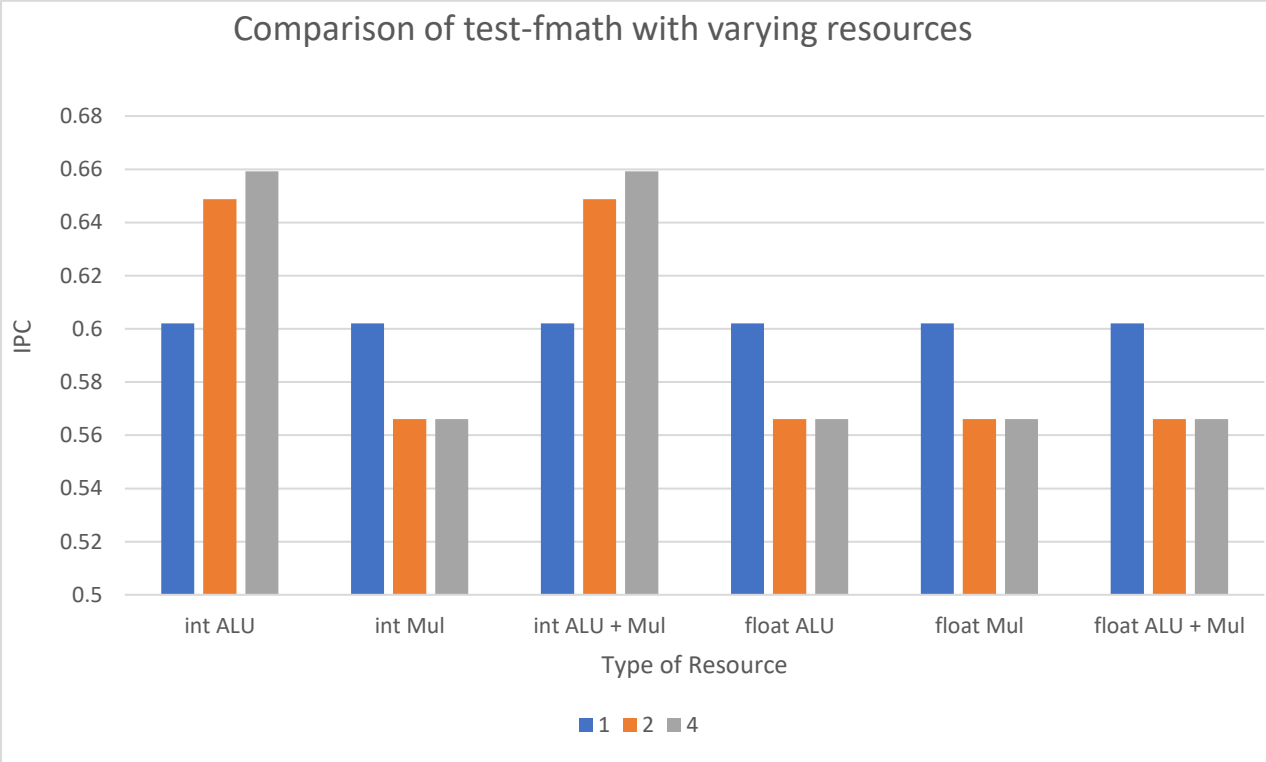
Ans – a) Initial config_a.cfg file after setting all resources values to 1

```
# run pipeline with in-order issue
-issue:inorder                false

# issue instructions down wrong execution paths
-issue:wrongpath               true

# register update unit (RUU) size (16 default)
-ruu:size                        8

# load/store queue (LSQ) size
-lsq:size                        8
```

```
# total number of integer ALU's available (4 default)
-res:ialu                        1

# total number of integer multiplier/dividers available
-res:imult                       1

# total number of memory system ports available (to CPU) (2 default)
-res:memport                     4

# total number of floating point ALU's available (4 default)
-res:fpalu                       1

# total number of floating point multiplier/dividers available
-res:fpmult                      1
```

# of resources		1	2	4
Integer ALU resource	test-math	0.6950	0.8167	0.8346
	test-fmath	0.6021	0.6487	0.6592
Integer multiplication resource	test-math	0.6950	0.6809	0.6809
	test-fmath	0.6021	0.5661	0.5661
Integer addition + multiplication resource	test-math	0.6950	0.8167	0.8346
	test-fmath	0.6021	0.6487	0.6592
Floating ALU resource	test-math	0.6950	0.6809	0.6809
	test-fmath	0.6021	0.5661	0.5661
Floating multiplication resource	test-math	0.6950	0.6809	0.6809
	test-fmath	0.6021	0.5661	0.5661
Floating addition + multiplication resource	test-math	0.6950	0.6809	0.6809
	test-fmath	0.6021	0.5661	0.5661


Comparison of test-math with varying resources
IPC
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
int ALU
int Mul
int ALU + Mul
float ALU
float Mul
float ALU + Mul
Type of Resource
1
2
4


Comparison of test-fmath with varying resources
IPC
0.68
0.66
0.64
0.62
0.6
0.58
0.56
0.54
0.52
0.5
int ALU
int Mul
int ALU + Mul
float ALU
float Mul
float ALU + Mul
Type of Resource
1
2
4

Observation: We observe that only increase integer ALU increases the IPC both in case of test-math and incase of test-fmath. Moreover increasing integer multiplication and floating resouces we observe a decrease in IPC instead of increase.

Explanation: One possible reason behind no increase in performance other than when integer ALU is increase might be because of the fact that integer ALU operations are bottlenecks in both codes. Hence, even if we were to parallelize the other parts of computation we still have to wait to integer computations to be done. This result directly comes from Amdahl's Law.

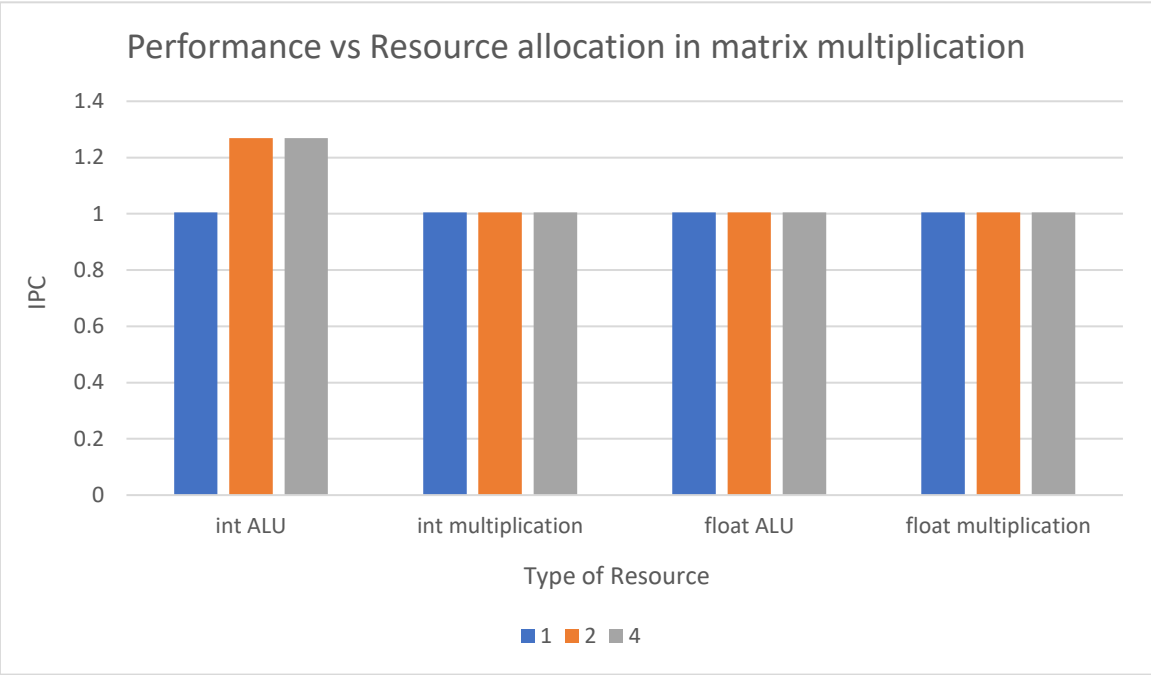
Reason behind decrease in IPC when say, when integer multiply is increased might be because even though integer ALU remains a bottleneck, assigning extra resources to integer multiply results in extra scheduling time and extra overhead with no increase in performance gained due to resources increase.

Ans – b) Code for matrix multiplication –

matmul.c

```c
void main(){
    int n=32;
    float A[n][n], B[n][n], C[n][n];
    // Randomly assigning matrices A and B values
    int i,j,k;
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            A[i][j] = (i+7.0)/(j+11);
            B[i][j] = (j+19.0)/(i+13);
        }
    }
    // Multiply
    for(i=0;i<n;i++){
        for(k=0;k<n;k++){
            float C_ik = 0;
            for(j=0;j<n;j++){
                C_ik += A[i][j]*B[j][k];
            }
            C[i][k] = C_ik;
        }
    }
}
```

# of resources	1	2	4
Integer ALU resource	1.0056	1.2689	1.2690
Integer multiplication resource	1.0056	1.0056	1.0056
Floating ALU resource	1.0056	1.0056	1.0056
Floating multiplication resource	1.0056	1.0056	1.0056



Observation: Similar to last case (a) we again observer that changing integer ALU's from 1 -> 2 has significant increase in IPC. Changing it again from 2 -> 3 has slight increase.

But once again changing any other type of resources allocated has no impact on IPC.

Explanation: To explain this behavior for all cases we run sim-profiling of the codes to check the integer computations and floating points computations –

```
sim_inst_class_prof.start_dist
load                  34096  15.96
store                 22794  10.67
uncond branch          9005   4.22
cond branch           29564  13.84
int computation      116257  54.42
fp computation         1879   0.88
trap                     29   0.01
sim_inst_class_prof.end_dist
```

test-math

```
sim_inst_class_prof.start_dist
load                   8623  16.15
store                  7712  14.45
uncond branch          2263   4.24
cond branch            8054  15.09
int computation       26653  49.93
fp computation           60   0.11
trap                     18   0.03
sim_inst_class_prof.end_dist
```

test-fmath

```
sim_inst_class_prof.start_dist
load                  488126  32.31
store                 213434  14.13
uncond branch          36089   2.39
cond branch            36599   2.42
int computation       660611  43.73
fp computation         75776   5.02
trap                       7   0.00
sim_inst_class_prof.end_dist
```

matrix-multiplication

In all of these cases the %age of floating points instructions executed are very low - 0.88% , 0.11% and 5.02% as compared to 54.42%, 49.93% and 43.73% of integer instructions executed.

As a result of this only increasing int ALU's helps because it remains a bottleneck and has over 10 times more instructions to be executes than floating operations.